STACKD STUDIOS AI

FREE RESOURCE · v2026.10

```
$ pre-launch --check
```

BEFORE YOU LAUNCH.

A security and readiness checklist for apps built with Claude Code, Cursor, Lovable, Bolt, and similar AI tools.

BUILD IT. FIX IT. SCALE IT.

01 / HOW TO USE THIS

- **Only have ten minutes?** Do the Top 5 on the next page.
- **Have an hour?** Work through sections 1 to 8 in order. Skip section 7 if your app doesn't take payments.
- Tick a box only when you've checked it yourself. "My AI tool said it's done" doesn't count yet.

02 / WHAT THE TAGS MEAN

FIX BEFORE LAUNCH

Don't go live until this is sorted.

FIX SOON

Fix it in your first week or two.

03 / FIVE WORDS YOU'LL SEE A LOT

Frontend: the part of your app that runs in the visitor's browser. Anyone can view its code.

API key: a password that lets your app use a service like Stripe, Supabase, or OpenAI.

Repo: your project's code folder on GitHub (or similar).

Backend / server: the part that runs on your hosting. Visitors can't see it.

Environment variable: a setting (often a key) stored on your host instead of written in your code.

> 00 / START HERE

TOP 5 CHECKS FIRST 10 MIN

Got ten minutes? Do these five. They cover the problems that tend to cause the most damage in AI-built apps.

01

☐ **Are your secret keys kept out of your frontend code and your GitHub repo?**

FIX BEFORE LAUNCH

WHY — Anyone can read code that runs in a browser. A leaked secret key can let a stranger use your paid accounts or read your data.

QUICK CHECK — Search your project for `sk_`, `secret`, `service_role`, and `API_KEY`. If any show up in frontend files or in GitHub, go to Section 1.

02

☐ **Is your database locked so people only see what they're allowed to?**

FIX BEFORE LAUNCH

WHY — Tools like Supabase and Firebase let the browser talk to your database directly. Without access rules, anyone can read or change your tables.

QUICK CHECK — Supabase: every table has row-level security (RLS) turned on. Firebase: your rules don't say `allow read, write: if true`. Details in Section 2.

03

☐ **Can each user see only their own data?** FIX BEFORE LAUNCH

WHY — This is one of the most common gaps in AI-built apps, and users notice fast.

QUICK CHECK — The two-account test. Make two accounts. Create something as Account A. Log in as Account B and try to open A's item by pasting its link. You should get an error or nothing.

04

☐ **Do private pages and admin pages need a login that the server checks?**

FIX BEFORE LAUNCH

WHY — Hiding a button doesn't protect the data behind it.

QUICK CHECK — Open a private browser window, log out, and paste the links to your dashboard and admin pages. They should send you to login without showing any data, even for a second.

05

☐ **Is your live site using live settings?** FIX BEFORE LAUNCH

WHY — Test keys, debug mode, or public preview links wired to real data are easy to miss.

QUICK CHECK — In your host (Vercel, Netlify, or similar), confirm production has its own keys set, debug mode is off, and preview links don't use your real database. Details in Section 6.

> 01 / SECRETS

SECRETS AND API KEYS

Some keys are meant to be public. Others are passwords. Knowing which is which is half the job.

USUALLY SAFE IN THE FRONTEND	NEVER IN THE FRONTEND
Stripe publishable key (`pk_live_` , `pk_test_`)	Stripe secret key (`sk_`) and restricted key (`rk_`)
Supabase anon or publishable key (safe only with RLS on)	Supabase `service_role` or secret key
Firebase web config (safe only with strict rules)	Stripe webhook signing secret (`whsec_`)
	OpenAI, Anthropic, and other AI API keys

1.1 Are all secret keys kept off the frontend? FIX BEFORE LAUNCH

WHY Code that runs in the browser can be read by anyone using the browser's developer tools.

HOW TO CHECK

1. Search your project for `sk_` , `rk_` , `whsec_` , `service_role` , `sb_secret_` , and the names of your AI providers.
2. Any variable starting with `NEXT_PUBLIC_` , `VITE_` , `REACT_APP_` , or `EXPO_PUBLIC_` is sent to the browser. Secret keys must never use these prefixes.
3. On your live site, right-click > View Page Source, then search for `sk_` . You should find nothing.

GOING WRONG An OpenAI key saved as `VITE_OPENAI_KEY` . Anyone can copy it and run up your bill.

THE FIX Move that call into a server function (an API route, a Supabase Edge Function, or a serverless function) and keep the key there.

1.2 Is your `.env` file kept out of GitHub? FIX BEFORE LAUNCH

WHY Deleting a file later doesn't remove it from your project's history. Old commits still hold it.

HOW TO CHECK

1. Open `.gitignore` and confirm it includes `.env` (or `.env*`).
2. On GitHub, search your repo's files for `.env` .
3. In your terminal, run `git log --all -- .env` . Any result means it was committed at some point.

GOING WRONG You deleted `.env` last week, but your very first commit still shows every key.

1.3 Have you replaced any key that was ever exposed? FIX BEFORE LAUNCH

WHY Removing a leaked key from your code doesn't make it safe. Assume someone already copied it.

HOW TO CHECK List every key that was ever in your code, a public repo, a screenshot, or a shared chat. For each one: create a new key in the provider's dashboard, update it in your host's environment variables, redeploy, then delete the old key.

1.4 Do your keys have limits? FIX SOON

WHY — If a key does leak, limits keep the damage small.

HOW TO CHECK — Set spending limits or billing alerts with your AI providers. In Stripe, consider restricted keys that only allow what your app needs.

> 02 / DATABASE

DATABASE ACCESS AND PERMISSIONS

Many AI tools build on Supabase or Firebase, where the browser talks to the database directly. That's fine, as long as rules decide who can see and change what.

2.1 Is row-level security (RLS) on for every table? (Supabase) FIX BEFORE LAUNCH

WHY — Your Supabase anon key lives in your frontend. With RLS off, anyone who copies it can read or change that table.

HOW TO CHECK

1. In Supabase, open Advisors > Security Advisor. It flags tables with RLS turned off.
2. Or open each table in the Table Editor and look for an RLS warning.
3. To turn it on, ask your AI tool, or run: `alter table your_table enable row level security;`

HEADS-UP — RLS with no policies blocks everything, so parts of your app may stop working. That's expected. Add policies next (2.2).

FIREBASE — Open Firestore > Rules. Watch for `allow read, write: if true;` or test-mode rules with an expiry date.

2.2 Do your policies let people see and change only their own rows? FIX BEFORE LAUNCH

WHY — RLS that's on but allows everyone is the same as RLS that's off.

HOW TO CHECK

1. In Supabase, open Authentication > Policies and read each one.
2. Look for policies that just say `true`. On private data, they're a red flag.
3. A safer pattern looks like `auth.uid() = user_id` ("only the logged-in owner").
4. Check reading, adding, editing, and deleting separately. Each can have its own policy.

GOING WRONG — A `profiles` table with an "Enable read access for all users" policy. Every user's email and phone number is readable by anyone.

2.3 Are file storage buckets private unless they're meant to be public? FIX BEFORE LAUNCH

WHY — Files in a public bucket can be opened by anyone who has or guesses the link.

HOW TO CHECK — Supabase: Storage, look for buckets marked Public. Firebase: Storage > Rules. Uploads like IDs, invoices, or private photos belong in private buckets.

2.4 Is your admin database key used only on the server? FIX BEFORE LAUNCH

WHY — The Supabase `service_role` (or secret) key skips RLS completely.

HOW TO CHECK — Search for `service_role` and `sb_secret_`. They should appear only in server code and in your host's environment variables, never with a public prefix.

> 03 / AUTH

LOGIN AND AUTHENTICATION

A login screen is only the front door. What matters is whether every room behind it checks who you are.

3.1 Does the server check who's logged in, not just the page? FIX BEFORE LAUNCH

WHY

Redirecting a page or hiding a button doesn't protect data. People can send requests to your server directly.

HOW TO CHECK

1. Ask your AI tool: "List every API route and server function. For each one, show whether it checks the logged-in user."
2. Anything that reads or changes private data should check.
3. Logged out, paste your private page links into a private browser window.

GOING WRONG

`/dashboard` sends you to login, but `/api/orders` hands every order to anyone who asks.

3.2 Does changing an ID in a link keep you out of other people's things? FIX BEFORE LAUNCH

WHY

Many apps put IDs in links (like `/invoice/1042`). If changing `1042` to `1043` shows someone else's invoice, every record is exposed.

HOW TO CHECK

Use the two-account test from the Top 5. Also try changing the numbers or IDs in your links while logged in.

3.3 Are admin features checked on the server? FIX BEFORE LAUNCH

WHY

If "is this person an admin?" is decided in the browser, or stored somewhere users can edit, anyone can promote themselves.

HOW TO CHECK

1. Logged in as a normal user, visit `/admin` and any admin links.
2. In Supabase, don't store roles in `user_metadata`, because users can update it themselves. Use `app_metadata` or a roles table protected by RLS.

3.4 Do sign-up, login, password reset, and email confirmation work on your live domain? FIX BEFORE LAUNCH

WHY

Redirect links often still point to `localhost` from testing, so emails send people to a dead page.

HOW TO CHECK

1. Supabase: Authentication > URL Configuration. Set the Site URL and Redirect URLs to your live domain.
2. Firebase: Authentication > Settings > Authorized domains.
3. Do the whole flow on the live site with a real email address.
4. Supabase's built-in email sender is meant for testing and has low limits. Set up your own email provider (SMTP) before launch.

☐ **3.5 Is there protection against repeated login attempts and fake sign-ups?** FIX SOON

WHY — Bots can guess passwords or flood your app with fake accounts.

HOW TO CHECK — Review your auth provider's rate limits, and consider adding a CAPTCHA (such as Cloudflare Turnstile or hCaptcha) to sign-up and login.

> 04 / INPUT

INPUT VALIDATION AND FORMS

Every form is an open door for whatever people type into it. Your app should decide what's allowed, not the visitor.

☐ **4.1 Is input checked on the server, not just in the form?** FIX BEFORE LAUNCH

WHY — Form rules (like "required" or "max 100 characters") run in the browser and can be skipped.

HOW TO CHECK
1. Ask your AI tool: "Where is each form's input validated on the server?" A library like Zod is a common way to do it.
2. Try empty fields, very long text, and negative numbers in quantity or price fields.

GOING WRONG — A quantity field that accepts `-5` and quietly turns an order into a credit.

☐ **4.2 Is text people type shown as text, not run as code?** FIX BEFORE LAUNCH

WHY — If your app displays someone's input as live code, an attacker can run scripts in other users' browsers. This is called cross-site scripting (XSS).

HOW TO CHECK
1. Type `<b>hello</b>` into a name or comment field and save it. If it shows up in bold, flag it.
2. Search your code for `dangerouslySetInnerHTML`, `innerHTML`, and `v-html`. Each one should clean the input first.
3. Ask your AI tool whether any database queries are built by gluing user text into SQL. They shouldn't be.

☐ **4.3 Are file uploads limited by type and size?** FIX SOON

WHY — Without limits, people can upload huge files or file types your app shouldn't accept.

HOW TO CHECK — Try uploading a large file and a file type you don't expect. Both should be rejected, and the limit should be enforced on the server or in your storage rules.

☐ **4.4 Do error messages stay short and vague for users?** FIX SOON

WHY — Detailed errors can reveal table names, file paths, or code.

HOW TO CHECK — Trigger an error on purpose (submit a broken form). Users should see a friendly message. The details belong in your logs.

4.5 Are public forms and AI features protected from abuse? FIX SOON

WHY — Bots fill contact forms with spam. If your app calls an AI API, a bot can also run up your bill.

HOW TO CHECK — Add a CAPTCHA or a hidden "honeypot" field to public forms. Limit how many AI requests each user can make per hour or day.

> 05 / PACKAGES

DEPENDENCIES AND PACKAGES

Packages are code other people wrote that your app relies on. AI tools add them freely, so it's worth knowing what's in there.

5.1 Have you run a security check on your packages? FIX SOON

WHY — Older package versions can have known security holes. Treat any "critical" or "high" result as fix before launch.

HOW TO CHECK

1. In your project folder, run `npm audit` in the terminal.
2. Deal with "critical" and "high" results first. `npm audit fix` updates what it safely can.
3. Avoid `npm audit fix --force` unless you're ready to test everything afterward. It can install big version jumps that break your app.
4. On GitHub, turn on Dependabot alerts in your repo's security settings so you hear about new issues.

5.2 Is every package real and well known? FIX BEFORE LAUNCH

WHY — AI tools sometimes suggest packages that don't exist. Security researchers have warned that attackers can publish harmful packages under those made-up names.

HOW TO CHECK — Open `package.json`. Look up any name you don't recognize on npmjs.com. Check the spelling, weekly downloads, last update, and whether it links to a real code repo.

5.3 Have you removed packages you don't use? FIX SOON

WHY — Every package is code you're trusting. Fewer means less to worry about.

HOW TO CHECK — Ask your AI tool to list packages in `package.json` that nothing imports, then remove them and test.

5.4 Is your lockfile committed? FIX SOON

WHY — The lockfile (`package-lock.json` , `pnpm-lock.yaml` , `yarn.lock` , or `bun.lockb`) makes every deploy install the exact same versions you tested.

HOW TO CHECK — Confirm the file exists in your GitHub repo and isn't listed in `.gitignore` .

> 06 / DEPLOY

DEPLOYMENT SETTINGS

Your app might work perfectly on your laptop and still be set up wrong online. Most hosts run several versions at once, so it's worth checking each.

- [ ] **6.1 Are your environment variables set in your host for production?** FIX BEFORE LAUNCH

  WHY — Your live site needs its own keys and settings, stored on the host instead of in your code.

  HOW TO CHECK
  1. Vercel: Project > Settings > Environment Variables. Each variable can be set for Production, Preview, or Development.
  2. Netlify: Site configuration > Environment variables, with separate values per deploy context.
  3. Redeploy after changing anything. Changes don't apply to sites that are already deployed.

- [ ] **6.2 Are preview deployments kept away from real data?** FIX BEFORE LAUNCH

  WHY — Hosts often create a public link for every branch or change. If those previews use your live database and keys, anyone with the link gets a test version connected to real data.

  HOW TO CHECK
  1. Give Preview its own variables, pointing to a test database and Stripe test keys.
  2. Check whether your host can password-protect previews, and turn it on.

- [ ] **6.3 Is debug mode off and test material cleaned up?** FIX BEFORE LAUNCH

  WHY — Debug settings can reveal internal details. Leftover test accounts can be easy ways in.

  HOW TO CHECK — Confirm production isn't in development or debug mode. Delete test users and sample data, and remove any "temporary" admin accounts or shortcuts. Ask your AI tool to find code that logs user data or passwords to the console.

- [ ] **6.4 Does your site load over HTTPS on your real domain?** FIX BEFORE LAUNCH

  WHY — HTTPS protects what people send you, like passwords and payment details.

  HOW TO CHECK — Vercel and Netlify set up HTTPS automatically. Type `http://` plus your domain and make sure it switches to `https://`. Check both the `www` and non-`www` versions.

- [ ] **6.5 Will you know if something breaks?** FIX SOON

  WHY — It's better to find errors before your users email you about them.

  HOW TO CHECK — Know where your host's logs are. Consider an error tracker (such as Sentry) and a free uptime check that alerts you if the site goes down.

> 07 / PAYMENTS

PAYMENTS AND WEBHOOKS

Skip this section if your app doesn't take payments. The examples use Stripe, but the same ideas apply to PayPal, Lemon Squeezy, and others. A **webhook** is a message your payment provider sends to your server, like "this payment went through."

- [ ] **7.1 Is the price decided on the server?** FIX BEFORE LAUNCH

  WHY — If the browser tells your server how much to charge, someone can change the amount.

  HOW TO CHECK — Ask your AI tool: "Where does the checkout amount come from?" The server should look up the price (for example, a Stripe Price ID) itself, not accept a number from the page.

  GOING WRONG — A $99 plan bought for $1 because the price was sent from the browser.

- [ ] **7.2 Does your webhook check Stripe's signature?** FIX BEFORE LAUNCH

  WHY — Your webhook address is public. Without a signature check, anyone can send a fake "payment succeeded" message.

  HOW TO CHECK — Your webhook code should use `stripe.webhooks.constructEvent` with your `whsec_` signing secret, stored as an environment variable. Ask your AI tool to confirm it does.

- [ ] **7.3 Do you give access only after the webhook confirms payment?** FIX BEFORE LAUNCH

  WHY — People can open your "success" page without paying, or close the tab before it loads.

  HOW TO CHECK — Find where your app unlocks the product or marks an order paid. It should happen in the webhook handler, not on the success page.

- [ ] **7.4 Are live and test keys in the right places?** FIX BEFORE LAUNCH

  WHY — Test keys on your live site mean no real money comes in. Live keys on previews mean real charges during testing.

  HOW TO CHECK — Production uses `pk_live_`, `sk_live_`, and the live webhook secret. Previews use test keys. In Stripe, live mode needs its own webhook endpoint, separate from test mode.

- [ ] **7.5 Have you tested a purchase, a failed card, and a cancellation or refund?** FIX BEFORE LAUNCH

  WHY — The happy path usually works. The others are where things break.

  HOW TO CHECK — In test mode, use Stripe's test cards (`4242 4242 4242 4242` succeeds, `4000 0000 0000 0002` is declined). Confirm each case updates your app correctly. Stripe can send the same event more than once, so make sure a repeat doesn't create a duplicate order.

> 08 / FINAL TEST

FINAL PRE-LAUNCH TEST

Set aside an hour, on the live site, ideally with a friend who's never seen your app.

- [ ] **New user test**
  Sign up with a brand-new email and complete the main thing your app does, start to finish.
- [ ] **Two-account test**
  Account B can't see or change anything that belongs to Account A.
- [ ] **Logged-out test**
  Private pages and admin pages show nothing without a login.
- [ ] **Phone test**
  Try it on a real phone, on mobile data, not just your laptop's Wi-Fi.
- [ ] **Break-it test**
  Submit empty forms, paste very long text, double-click every submit button, and hit Back halfway through checkout.
- [ ] **Backup test**
  You know where your database backups are and how to restore one. Check what your plan includes, because some free plans don't keep backups you can download.
- [ ] **Emergency plan**
  You know how to take the site offline and how to replace your keys quickly if something leaks.
- [ ] **AI second look**
  Paste the prompt below into your AI coding tool.

$ paste into your AI coding tool

```
Review this project for security issues before launch. Check for: secret keys in frontend
code or committed files, database tables without row-level security or with overly open
policies, API routes that don't check the logged-in user, missing server-side input
validation, and payment webhooks without signature verification. List each finding with the
file, how serious it is, and how to fix it. Don't change any code yet.
```

An AI review is a helpful extra pair of eyes, not a guarantee. Read its findings carefully and test the fixes yourself.

> 09 / WHAT'S NEXT

YOU DON'T HAVE TO DO IT ALONE.

STUCK ON SOMETHING?

Ask in our Discord: **discord.gg/BvkWgVRSn**

Post in `#help-desk` with what you're building, the tool you used, and what you've tried. No question is too basic here.

STACKD RESCUE

BUILT FAST. BROKE LATER. WE FIX THAT.

Want us to fix, secure, or finish your app?

REQUEST A REVIEW >

`dash.stackdstudiosai.com/forms/rescue`

01 / AUDIT
We look at what you have.

02 / RATE
Every finding gets a severity.

03 / PLAN
You get a plan and a quote before we touch any code.

FOUNDING MEMBERS

The first founding members of our Discord can get a **free quick look at one app**: a short written summary of the biggest security issues and obvious problems. Fixes aren't included; those go through a Rescue review. To claim yours, ask in `#stackd-rescue`.

This is a general checklist, not a security audit or a guarantee that your app is safe. Tools and settings change often, so check your provider's current docs.

STACKD STUDIOS AI